

DOMAIN 1

AGENT ARCHITECTURE & ORCHESTRATION

Complete deep-dive for this blueprint domain (27% of exam) of the Claude Certified Architect — Foundations exam: every concept with worked examples, a dedicated traps-and-nuances section explaining why the tempting wrong answers are wrong, a one-page cheat sheet, and a full bank of 138 original practice questions with answers and rationales.

Section	What's inside
1. Cheat Sheet	Golden rule, quick-reference tables, and a scenario decision checklist — read this first and last.
2. Concept Guide	Full narrative coverage of every concept tested in this domain, with worked scenarios.
3. Traps & Nuances	18 recurring wrong-answer patterns: the trap, why it's tempting, and the correct principle.
4. Practice Questions	138 original scenario-based questions, 4 options each, answer marked, rationale included.

This is an independent study aid, not an official Anthropic publication or a reproduction of actual exam questions, which Anthropic does not publish.

1 · CHEAT SHEET

Golden rule: the coordinator owns scoping, delegation, error handling, and synthesis; subagents own only narrow execution of exactly what they were assigned. Most root-cause questions in this domain resolve to “check the coordinator’s decomposition first.”

stop_reason quick map

tool_use	Execute tool(s), return results, continue loop.
end_turn	Stop — genuine completion.
max_tokens	Truncated — NOT complete. Don’t act on it as final.
stop_sequence	Configured stop string hit — deliberate, not an error.

Topology cheat sheet

Hub-and-spoke	Default choice. Central coordinator; independent subagents.
Pipeline	Strict sequential handoff, stage N output feeds stage N+1.
Peer-to-peer	Decentralized negotiation only — highest risk, rarely the right default.

Reliable vs. unreliable escalation triggers

Reliable	Explicit user request · genuine policy gap · demonstrated inability to progress
Unreliable	Sentiment analysis · self-reported model confidence · soft classifier scores

Enforcement: hook vs. prompt

Hard policy (money/safety/legal)	PreToolUse hook — deterministic, fires every time.
Soft preference / style	System prompt instruction is fine — probabilistic is acceptable here.

Decision checklist for any Domain 1 scenario

- Multi-agent result incomplete/skewed but each subagent looks correct? → check the coordinator’s decomposition first.
- Subagent missing information it “should have known”? → check whether the coordinator explicitly passed it — nothing is inherited automatically.
- Choosing parallel vs. sequential? → independent = parallel; genuine dependency = sequential, never the reverse.
- A rule must always hold? → hook/programmatic, not prompt wording.
- Deciding whether to escalate? → explicit request, genuine policy gap, or demonstrated stuck-ness only.
- Loop hit its iteration cap? → treat as a distinct failure mode, never as silent success.

Agent Architecture & Orchestration

2 · CONCEPT GUIDE

This is the largest domain and the one candidates most often underestimate. It covers the mechanics of the agentic loop, when to reach for agents at all, how to decompose and delegate work, multi-agent topology, and where policy enforcement belongs.

1.1 What makes a system "agentic"

An agentic system is defined by three properties working together: the model has meaningful **autonomy** over what step to take next, it can invoke **tools** to change or observe the world beyond the conversation, and it operates inside a **loop** that keeps running until a task-defined stopping condition is met — not a fixed number of turns. A single-turn conversational exchange or a fixed linear pipeline (call model, call one API, format, return) is not agentic even if it uses the same underlying model. The exam distinguishes three tiers:

- **Conversational system** — a human drives every turn; the model never decides to act unprompted.
- **Workflow** — a fixed, human-authored sequence of steps (possibly using the LLM inside a step) with no runtime branching driven by the model's own judgment.
- **Agent** — the model itself decides which tools to call, in what order, and when it is done, adapting that plan as new information arrives.

When to use agentic architecture vs. a simpler pattern

The exam rewards restraint. Agentic architecture adds latency, cost, and failure surface, so it is justified when the task has open-ended scope, requires multiple dependent tool calls whose order can't be fixed in advance, or needs the system to adapt its plan based on intermediate results. A workflow (or even a single well-structured prompt) is the right call when the steps are fully knowable ahead of time — a fixed ETL pipeline with an LLM classification step in the middle does not need to be an agent.

1.2 The agentic loop mechanics

The Claude API surfaces the loop state through the **stop_reason** field on a response:

```
stop_reason: "tool_use" -> the model wants to call one or more tools.
Execute them, return tool_result blocks, and
send the conversation back to the model.
stop_reason: "end_turn" -> the model has produced its final answer.
Stop the loop.
stop_reason: "max_tokens" -> the response was truncated. Do not treat
this as completion.
stop_reason: "stop_sequence" -> a configured stop string was hit.
```

The exam explicitly tests two anti-patterns here, both of which look reasonable at first glance:

- ✗ Determining task completion by parsing the assistant's natural-language text for phrases like "I'm done" instead of checking **stop_reason** programmatically.
- ✗ Capping the loop at an arbitrary fixed iteration count and treating that as "completion" rather than a safety ceiling that should trigger explicit escalation when hit.

✓ Loop on **stop_reason == tool_use**, stop cleanly on **end_turn**, and treat a hit iteration ceiling as a distinct failure mode to surface, not a silent success.

1.3 Task decomposition & delegation

Good decomposition splits a task along boundaries that map to independently verifiable subtasks with minimal shared state. Bad decomposition splits along the first structure that comes to mind (e.g., “three examples”) without checking whether that structure actually covers the requested scope.

Worked failure scenario (high-yield for the exam)

A coordinator is asked to research “AI’s impact on creative industries.” It decomposes the task into three subagent assignments: AI in digital art, AI in graphic design, and AI in photography. Each subagent does excellent, accurate work. The final synthesized report, however, only covers visual art — it never touches music, writing, film, or performance. Where is the fault?

Root cause: the coordinator, not the subagents and not the synthesis step. The subagents did exactly what they were assigned. The coordinator silently narrowed a broad topic to a convenient subset at decomposition time and never assigned anything else. This is the single most representative failure pattern in the domain: garbage-in decomposition produces confidently correct, incompletely scoped output, and every downstream component looks like it worked.

Lesson for the exam: when a scenario asks you to find the root cause of an incomplete or skewed multi-agent result, check the **decomposition step first**, before blaming synthesis, prompting, or the subagents’ tool use.

1.4 Multi-agent topology

- **Hub-and-spoke (orchestrator–subagent)** — the default, exam-favored pattern. A coordinator owns task decomposition, delegation, error handling, and result aggregation. Subagents execute in **isolated context**: they inherit nothing automatically from the coordinator’s conversation. Anything a subagent needs — facts, constraints, prior findings, formatting requirements — must be explicitly packed into its task prompt, or it effectively does not exist for that subagent.
- **Pipeline** — sequential handoff, output of stage N becomes input to stage N+1. Good for well-ordered transformations (extract → validate → summarize); poor fit when steps need to run concurrently or revisit earlier stages.
- **Peer-to-peer** — agents communicate directly without a central coordinator. Highest flexibility, highest risk of duplicated work, race conditions, and unbounded message loops; requires the most deliberate guardrails and is rarely the exam’s “correct” answer unless the scenario explicitly calls for decentralized negotiation.

Context isolation is the load-bearing idea in this section. Because subagents don’t share memory with the coordinator or each other by default, coordinator prompts must explicitly restate relevant facts, and subagent context budgets must be sized for what that subagent alone needs — not the whole system’s context.

1.5 Sequential vs. parallel execution, planning & replanning

Run subagents in parallel when their subtasks are independent and don’t need each other’s intermediate output (e.g., three independent research streams). Run sequentially when a later step genuinely depends on an earlier step’s result (e.g., validate before you write). Parallelizing dependent steps is a common exam distractor — it looks like a performance win but silently drops the dependency.

Dynamic planning means the coordinator can revise its plan mid-execution when a subagent returns an unexpected result, a tool fails, or new information changes scope. A system that commits to a fixed plan up front and executes it blind to intermediate results handles ambiguity poorly; the exam favors coordinators that treat the plan as a living object, re-evaluated after each major step.

1.6 Escalation, ambiguity handling & policy enforcement

This subsection overlaps with Domain 5 but the exam also tests it from the orchestration angle: **where** a policy is enforced matters as much as what the policy says.

Worked scenario

A support agent must never process a refund over £500 without human approval. Two implementations are proposed: (a) a system-prompt instruction telling the model not to process refunds above the threshold, or (b) a **PreToolUse** hook that programmatically blocks the **process_refund** tool call whenever the amount exceeds £500, before it ever reaches execution.

A system-prompt instruction is probabilistic — it holds *most of the time*, and the failure mode is silent (the model just complies with an edge-case framing it wasn't trained to resist). A hook is deterministic — it fires on *every* call regardless of how the model was talked into requesting it. When the cost of a miss is financial, legal, or safety-critical, the exam wants the deterministic, programmatic control, not a stronger-worded prompt.

- Reliable, exam-favored controls: **hooks** (PreToolUse/PostToolUse), schema validation, code-level checks, allow/deny lists enforced outside the model.
- Unreliable controls the exam uses as distractors: relying solely on system-prompt wording, self-reported model confidence, or “please double-check yourself” instructions for hard policy limits.

1.7 Orchestrator-subagent responsibilities at a glance

Responsibility	Owner
Task decomposition & scoping	Coordinator
Delegation & context packing per subagent	Coordinator
Executing a narrow, well-scoped subtask	Subagent
Error handling / retry policy across subagents	Coordinator
Result aggregation & synthesis	Coordinator
Escalation to a human	Coordinator (via explicit, reliable triggers — see Domain 5)

3 · TRAPS & NUANCES

Each card below names a specific concept, describes the wrong answer the exam is likely to dangle in front of you, explains why that wrong answer feels right, and states the principle that actually resolves it. Read this section right before the exam as a final refresh.

CONCEPT: stop_reason handling

The trap: Treating stop_reason: "max_tokens" the same as "end_turn" and acting on the response as if it were complete.

Why it's tempting: Both values mean generation stopped, and a truncated response can still look plausible and well-formed at a glance.

The correct principle: max_tokens means the output was cut off mid-generation — it must be treated as incomplete, not as a finished answer.

CONCEPT: Task completion detection

The trap: Parsing the assistant's natural-language text for phrases like "I'm done" or "task complete" to decide whether to stop looping.

Why it's tempting: It's the most intuitive way to "read" completion, and it often appears to work in casual testing.

The correct principle: Completion should be read from stop_reason programmatically — text-based detection breaks the moment phrasing varies.

CONCEPT: Iteration caps

The trap: Treating a hit iteration ceiling as if the task succeeded, since the loop technically "finished."

Why it's tempting: The loop did stop, and stopping is normally the signal for success.

The correct principle: A hit iteration cap is a safety ceiling, not a completion signal — it should trigger explicit escalation or failure handling.

CONCEPT: Decomposition root-cause diagnosis

The trap: Blaming the subagents or the synthesis step when a multi-agent result is incomplete or skewed.

Why it's tempting: The subagents' individual outputs are what's visibly wrong or missing in the final report, so they look like the source of the error.

The correct principle: If subagents did exactly what they were assigned, the fault is almost always in how the coordinator scoped the decomposition, not in execution or synthesis.

CONCEPT: Context isolation between coordinator and subagents

The trap: Assuming a subagent automatically "knows" something the coordinator discussed earlier in the same overall task.

Why it's tempting: It feels like one continuous conversation, so information seems like it should carry over naturally.

The correct principle: Subagents inherit nothing automatically; anything they need must be explicitly packed into their task prompt.

CONCEPT: Policy enforcement placement

The trap: Choosing a more firmly worded system-prompt instruction as the fix for a hard financial, legal, or safety policy.

Why it's tempting: It's the cheapest, most familiar fix, and it often does reduce the failure rate somewhat.

The correct principle: Hard policies need deterministic, programmatic enforcement (a hook, schema, or code check) — prompt wording is probabilistic and can be talked around.

CONCEPT: Escalation trigger reliability

The trap: Treating the model's self-reported confidence score as a legitimate signal for whether to escalate to a human.

Why it's tempting: A confidence score sounds quantifiable and objective, like a real measurement.

The correct principle: Self-reported confidence is a soft, internally-generated signal that can be poorly calibrated — reliable triggers are explicit requests or structurally verifiable conditions.

CONCEPT: Parallel vs. sequential execution

The trap: Parallelizing steps that actually have a real dependency, purely to “speed things up.”

Why it's tempting: Parallel execution is the intuitive lever for reducing latency, and the dependency isn't always obvious from the task description.

The correct principle: Only genuinely independent subtasks should run in parallel; a later step that needs an earlier step's result must run after it.

CONCEPT: Agent vs. workflow classification

The trap: Labeling any system that calls an LLM at some point as “agentic.”

Why it's tempting: The presence of a model call feels like the defining feature of an “AI system.”

The correct principle: Classification hinges on runtime autonomy: fixed steps with no model-driven branching is a workflow, not an agent, regardless of whether an LLM is involved.

CONCEPT: Multi-agent topology choice

The trap: Defaulting to a peer-to-peer topology because it “sounds” more flexible or advanced.

Why it's tempting: Decentralized, agent-to-agent negotiation sounds like the more sophisticated architecture.

The correct principle: Hub-and-spoke is the safer default for most tasks; peer-to-peer adds real coordination risk (duplicated work, race conditions) and should be reserved for genuine decentralized-negotiation scenarios.

CONCEPT: Dynamic replanning

The trap: Sticking rigidly to the original plan after new information emerges, in the name of “consistency.”

Why it's tempting: Consistency sounds disciplined and predictable, which feels like good engineering practice.

The correct principle: The plan should be treated as a living object — the coordinator should replan when intermediate results genuinely change what's known.

CONCEPT: Escalation ownership

The trap: Letting a subagent independently decide to escalate to a human mid-task.

Why it's tempting: The subagent is the one that “found” the problem, so it seems natural for it to act on that.

The correct principle: Escalation is a coordinator-level decision — the coordinator has the full task context needed to judge whether escalation is actually warranted.

CONCEPT: Cost and latency of agentic overuse

The trap: Adding more subagents to a slow system on the assumption that more compute or more parallel agents will speed it up.

Why it's tempting: “More agents working on it” intuitively feels like it should be faster, similar to adding more workers to a job.

The correct principle: Agentic overhead (coordination, context passing, synthesis) can make a system slower and more expensive than a simpler pattern for tasks that didn't need agentic architecture in the first place.

CONCEPT: Nested subagent recursion

The trap: Allowing subagents to spawn their own subagents with no depth limit, for architectural “elegance.”

Why it's tempting: Recursive delegation looks clean and composable in a diagram.

The correct principle: Unbounded recursive delegation can silently multiply cost and latency — depth should be explicitly capped with justification required to go deeper.

CONCEPT: Human-in-the-loop checkpoint placement

The trap: Placing a human “approval” step as a notification that fires after an irreversible action has already executed.

Why it's tempting: It still technically involves a human being informed, so it can look like a safeguard was included.

The correct principle: A human-in-the-loop checkpoint only has protective value if it can prevent the action — it must gate execution, not just report on it afterward.

CONCEPT: Observability across a multi-agent pipeline

The trap: Logging only the coordinator's final synthesized output, on the assumption that's “all that matters.”

Why it's tempting: The final output is what the user actually sees, so it feels like the only thing worth recording.

The correct principle: Without per-subagent traceability (task prompt, tool calls, result, linked by a trace ID), diagnosing which component introduced an error becomes guesswork.

CONCEPT: Ambiguity handling

The trap: Always stopping to ask a clarifying question, even for low-stakes ambiguity, to be “thorough.”

Why it's tempting: Asking feels safe and conscientious — better to check than to guess, intuitively.

The correct principle: For low-stakes ambiguity, stating a reasonable assumption and proceeding keeps the system useful; reserve clarifying questions for cases where a wrong guess is genuinely costly.

CONCEPT: Retry ownership across subagents

The trap: Letting each subagent retry its own failures independently with no coordinator visibility into the pattern.

Why it's tempting: It looks efficient and “self-healing” — each part handles its own problems.

The correct principle: Retry policy across subagents is a coordinator responsibility, since only the coordinator has the visibility to decide whether retrying, falling back, or escalating is the right call.

4 · PRACTICE QUESTIONS (138)

Q1. A model response returns stop_reason: "tool_use". What should the orchestration loop do?

- A. Stop the loop immediately.
- B. Treat the response as truncated and resend the identical prompt.
- C. Discard the response and ask the user to rephrase.
- D. ✓ Execute the requested tool call(s), return tool_result blocks, and continue the loop.

Correct answer: D. tool_use means the model wants a tool executed; run it, return the result, and keep looping.

Q2. A model response returns stop_reason: "end_turn". What should the orchestration loop do?

- A. Automatically retry with a longer prompt.
- B. Wait for a second stop_reason before stopping.
- C. ✓ Stop the loop — the model has produced its final answer.
- D. Execute any tool-shaped text found in the response.

Correct answer: C. end_turn signals genuine completion; continuing the loop afterward wastes calls and can cause drift.

Q3. A model response returns stop_reason: "max_tokens". What should the orchestration loop do?

- A. Restart the entire task from scratch automatically.
- B. ✓ Treat the response as truncated, not complete, and avoid acting on it as a final answer.
- C. Treat it exactly like end_turn.
- D. Silently discard the partial output with no follow-up.

Correct answer: B. max_tokens means generation was cut off; treating a truncated response as complete risks acting on partial output.

Q4. A model response returns stop_reason: "stop_sequence". What should the orchestration loop do?

- A. ✓ Recognize a configured stop string was hit and handle it per its intended purpose.
- B. Treat it as an unrecoverable error requiring escalation.
- C. Ignore it and keep generating anyway.
- D. Treat it identically to max_tokens.

Correct answer: A. stop_sequence is a deliberate, configured condition — not an error and not the same as truncation.

Q5. A coordinator is asked to research "AI's impact on creative industries" and decomposes it into subagent assignments covering only three visual-art subtopics (digital art, graphic design, photography). Each subagent does accurate work; the final report reads as complete but never touches music, writing, film, and performance. Where is the fault?

- A. The subagents, for not asking clarifying questions.
- B. The synthesis step, for not flagging the gap.
- C. The user, for an under-specified original prompt.
- D. ✓ The coordinator, which silently narrowed a broad topic to a convenient subset at decomposition time.

Correct answer: D. The subagents executed exactly what they were assigned; the coordinator narrowed "AI's impact on creative industries" to three visual-art subtopics (digital art, graphic design, photography) at decomposition time and never assigned coverage of music, writing, film, and performance. Root-cause the decomposition step first.

Q6. A coordinator is asked to research "global supply chain risk" and decomposes it into subagent assignments covering only shipping and logistics delays. Each subagent does accurate work; the final report reads as complete but never touches supplier concentration, geopolitical, and currency risk. Where is the fault?

- A. The synthesis step, for not flagging the gap.
- B. The user, for an under-specified original prompt.
- C. ✓ The coordinator, which silently narrowed a broad topic to a convenient subset at decomposition time.
- D. The subagents, for not asking clarifying questions.

Correct answer: C. The subagents executed exactly what they were assigned; the coordinator narrowed "global supply chain risk" to shipping and logistics delays at decomposition time and never assigned coverage of supplier concentration, geopolitical, and currency risk. Root-cause the decomposition step first.

Q7. A coordinator is asked to research "quarterly customer sentiment" and decomposes it into subagent assignments covering only NPS survey scores. Each subagent does accurate work; the final report reads as complete but never touches support-ticket sentiment and social-media mentions. Where is the fault?

- A. The user, for an under-specified original prompt.
- B. ✓ The coordinator, which silently narrowed a broad topic to a convenient subset at decomposition time.
- C. The subagents, for not asking clarifying questions.
- D. The synthesis step, for not flagging the gap.

Correct answer: B. The subagents executed exactly what they were assigned; the coordinator narrowed "quarterly customer sentiment" to NPS survey scores at decomposition time and never assigned coverage of support-ticket sentiment and social-media mentions. Root-cause the decomposition step first.

Q8. A coordinator is asked to research "competitive landscape analysis" and decomposes it into subagent assignments covering only pricing comparisons. Each subagent does accurate work; the final report reads as complete but never touches feature gaps, market share, and hiring trends. Where is the fault?

- A. ✓ The coordinator, which silently narrowed a broad topic to a convenient subset at decomposition time.
- B. The subagents, for not asking clarifying questions.
- C. The synthesis step, for not flagging the gap.
- D. The user, for an under-specified original prompt.

Correct answer: A. The subagents executed exactly what they were assigned; the coordinator narrowed "competitive landscape analysis" to pricing comparisons at decomposition time and never assigned coverage of feature gaps, market share, and hiring trends. Root-cause the decomposition step first.

Q9. A coordinator is asked to research "organizational security posture" and decomposes it into subagent assignments covering only network security. Each subagent does accurate work; the final report reads as complete but never touches endpoint, identity, and physical security. Where is the fault?

- A. The subagents, for not asking clarifying questions.
- B. The synthesis step, for not flagging the gap.
- C. The user, for an under-specified original prompt.
- D. ✓ The coordinator, which silently narrowed a broad topic to a convenient subset at decomposition time.

Correct answer: D. The subagents executed exactly what they were assigned; the coordinator narrowed "organizational security posture" to network security at decomposition time and never assigned coverage of endpoint, identity, and physical security. Root-cause the decomposition step first.

Q10. A coordinator is asked to research "employee engagement" and decomposes it into subagent assignments covering only exit-interview themes. Each subagent does accurate work; the final report reads as complete but never touches pulse-survey results and manager 1:1 notes. Where is the fault?

- A. The synthesis step, for not flagging the gap.
- B. The user, for an under-specified original prompt.
- C. ✓ The coordinator, which silently narrowed a broad topic to a convenient subset at decomposition time.
- D. The subagents, for not asking clarifying questions.

Correct answer: C. The subagents executed exactly what they were assigned; the coordinator narrowed "employee engagement" to exit-interview themes at decomposition time and never assigned coverage of pulse-survey results and manager 1:1 notes. Root-cause the decomposition step first.

Q11. A coordinator is asked to research "market entry strategy" and decomposes it into subagent assignments covering only regulatory requirements. Each subagent does accurate work; the final report reads as complete but never touches competitive intensity and customer demand signals. Where is the fault?

- A. The user, for an under-specified original prompt.
- B. ✓ The coordinator, which silently narrowed a broad topic to a convenient subset at decomposition time.
- C. The subagents, for not asking clarifying questions.
- D. The synthesis step, for not flagging the gap.

Correct answer: B. The subagents executed exactly what they were assigned; the coordinator narrowed "market entry strategy" to regulatory requirements at decomposition time and never assigned coverage of competitive intensity and customer demand signals. Root-cause the decomposition step first.

Q12. A coordinator is asked to research "product roadmap review" and decomposes it into subagent assignments covering only engineering feasibility. Each subagent does accurate work; the final report reads as complete but never touches customer demand and go-to-market readiness. Where is the fault?

- A. ✓ The coordinator, which silently narrowed a broad topic to a convenient subset at decomposition time.
- B. The subagents, for not asking clarifying questions.
- C. The synthesis step, for not flagging the gap.
- D. The user, for an under-specified original prompt.

Correct answer: A. The subagents executed exactly what they were assigned; the coordinator narrowed "product roadmap review" to engineering feasibility at decomposition time and never assigned coverage of customer demand and go-to-market readiness. Root-cause the decomposition step first.

Q13. A coordinator is asked to research "brand health" and decomposes it into subagent assignments covering only social-media sentiment. Each subagent does accurate work; the final report reads as complete but never touches press coverage, search trends, and customer surveys. Where is the fault?

- A. The subagents, for not asking clarifying questions.
- B. The synthesis step, for not flagging the gap.
- C. The user, for an under-specified original prompt.
- D. ✓ The coordinator, which silently narrowed a broad topic to a convenient subset at decomposition time.

Correct answer: D. The subagents executed exactly what they were assigned; the coordinator narrowed "brand health" to social-media sentiment at decomposition time and never assigned coverage of press coverage, search trends, and customer surveys. Root-cause the decomposition step first.

Q14. A coordinator is asked to research "vendor risk assessment" and decomposes it into subagent assignments covering only financial stability. Each subagent does accurate work; the final report reads as complete but never touches data-security posture and delivery-performance history. Where is the fault?

- A. The synthesis step, for not flagging the gap.
- B. The user, for an under-specified original prompt.
- C. ✓ The coordinator, which silently narrowed a broad topic to a convenient subset at decomposition time.
- D. The subagents, for not asking clarifying questions.

Correct answer: C. The subagents executed exactly what they were assigned; the coordinator narrowed "vendor risk assessment" to financial stability at decomposition time and never assigned coverage of data-security posture and delivery-performance history. Root-cause the decomposition step first.

Q15. A production system must guarantee that refunds above a fixed dollar threshold require human approval. Which enforcement mechanism is most reliable?

- A. Asking the model to double-check itself before calling the tool.
- B. ✓ A PreToolUse hook that programmatically blocks the disqualifying tool call before it executes.
- C. A firmly worded instruction in the system prompt.
- D. A note in CLAUDE.md asking the model to remember the rule.

Correct answer: B. Programmatic enforcement (a hook) fires deterministically on every call regardless of how the model was prompted; prompt wording alone is probabilistic and can be talked around at the edges.

Q16. A production system must guarantee that personally identifiable information must be redacted before any logging call. Which enforcement mechanism is most reliable?

- A. ✓ A PreToolUse hook that programmatically blocks the disqualifying tool call before it executes.
- B. A firmly worded instruction in the system prompt.
- C. A note in CLAUDE.md asking the model to remember the rule.
- D. Asking the model to double-check itself before calling the tool.

Correct answer: A. Programmatic enforcement (a hook) fires deterministically on every call regardless of how the model was prompted; prompt wording alone is probabilistic and can be talked around at the edges.

Q17. A production system must guarantee that a production deploy requires two independent approvals. Which enforcement mechanism is most reliable?

- A. A firmly worded instruction in the system prompt.
- B. A note in CLAUDE.md asking the model to remember the rule.
- C. Asking the model to double-check itself before calling the tool.
- D. ✓ A PreToolUse hook that programmatically blocks the disqualifying tool call before it executes.

Correct answer: D. Programmatic enforcement (a hook) fires deterministically on every call regardless of how the model was prompted; prompt wording alone is probabilistic and can be talked around at the edges.

Q18. A production system must guarantee that expense reports over \$1,000 require manager sign-off. Which enforcement mechanism is most reliable?

- A. A note in CLAUDE.md asking the model to remember the rule.
- B. Asking the model to double-check itself before calling the tool.
- C. ✓ A PreToolUse hook that programmatically blocks the disqualifying tool call before it executes.
- D. A firmly worded instruction in the system prompt.

Correct answer: C. Programmatic enforcement (a hook) fires deterministically on every call regardless of how the model was prompted; prompt wording alone is probabilistic and can be talked around at the edges.

Q19. A production system must guarantee that a prescribed medication dosage must never exceed the labeled maximum. Which enforcement mechanism is most reliable?

- A. Asking the model to double-check itself before calling the tool.
- B. ✓ A PreToolUse hook that programmatically blocks the disqualifying tool call before it executes.
- C. A firmly worded instruction in the system prompt.
- D. A note in CLAUDE.md asking the model to remember the rule.

Correct answer: B. Programmatic enforcement (a hook) fires deterministically on every call regardless of how the model was prompted; prompt wording alone is probabilistic and can be talked around at the edges.

Q20. A production system must guarantee that wire transfers above \$10,000 require secondary verification. Which enforcement mechanism is most reliable?

- A. ✓ A PreToolUse hook that programmatically blocks the disqualifying tool call before it executes.
- B. A firmly worded instruction in the system prompt.
- C. A note in CLAUDE.md asking the model to remember the rule.
- D. Asking the model to double-check itself before calling the tool.

Correct answer: A. Programmatic enforcement (a hook) fires deterministically on every call regardless of how the model was prompted; prompt wording alone is probabilistic and can be talked around at the edges.

Q21. A production system must guarantee that discount codes may never exceed a 30% markdown. Which enforcement mechanism is most reliable?

- A. A firmly worded instruction in the system prompt.
- B. A note in CLAUDE.md asking the model to remember the rule.
- C. Asking the model to double-check itself before calling the tool.
- D. ✓ A PreToolUse hook that programmatically blocks the disqualifying tool call before it executes.

Correct answer: D. Programmatic enforcement (a hook) fires deterministically on every call regardless of how the model was prompted; prompt wording alone is probabilistic and can be talked around at the edges.

Q22. A production system must guarantee that account deletion requires an explicit, separate confirmation step. Which enforcement mechanism is most reliable?

- A. A note in CLAUDE.md asking the model to remember the rule.
- B. Asking the model to double-check itself before calling the tool.
- C. ✓ A PreToolUse hook that programmatically blocks the disqualifying tool call before it executes.
- D. A firmly worded instruction in the system prompt.

Correct answer: C. Programmatic enforcement (a hook) fires deterministically on every call regardless of how the model was prompted; prompt wording alone is probabilistic and can be talked around at the edges.

Q23. A production system must guarantee that customer data may never be exported outside its home region. Which enforcement mechanism is most reliable?

- A. Asking the model to double-check itself before calling the tool.
- B. ✓ A PreToolUse hook that programmatically blocks the disqualifying tool call before it executes.
- C. A firmly worded instruction in the system prompt.
- D. A note in CLAUDE.md asking the model to remember the rule.

Correct answer: B. Programmatic enforcement (a hook) fires deterministically on every call regardless of how the model was prompted; prompt wording alone is probabilistic and can be talked around at the edges.

Q24. A production system must guarantee that API credentials must be rotated before being granted to a new service. Which enforcement mechanism is most reliable?

- A. ✓ A PreToolUse hook that programmatically blocks the disqualifying tool call before it executes.
- B. A firmly worded instruction in the system prompt.
- C. A note in CLAUDE.md asking the model to remember the rule.
- D. Asking the model to double-check itself before calling the tool.

Correct answer: A. Programmatic enforcement (a hook) fires deterministically on every call regardless of how the model was prompted; prompt wording alone is probabilistic and can be talked around at the edges.

Q25. Three subagents each research an independent, unrelated topic with no shared dependency. How should these subtasks be executed?

- A. Sequential — a later step depends on an earlier step's result.
- B. Peer-to-peer — the subagents should negotiate directly.
- C. Pipeline with no ordering constraints.
- D. ✓ Parallel — the subtasks are independent.

Correct answer: D. Independent subtasks with no shared dependency are strong candidates for parallel execution; a subtask that genuinely needs a prior step's output must run sequentially after it.

Q26. A validation subagent must confirm data quality before a writing subagent drafts the report. How should these subtasks be executed?

- A. Sequential — a later step depends on an earlier step's result.
- B. Peer-to-peer — the subagents should negotiate directly.
- C. ✓ Sequential — writing depends on validation's result.
- D. Parallel — the subtasks are independent.

Correct answer: C. Independent subtasks with no shared dependency are strong candidates for parallel execution; a subtask that genuinely needs a prior step's output must run sequentially after it.

Q27. One subagent fetches a stock price; a second subagent uses that price to compute portfolio value. How should these subtasks be executed?

- A. Peer-to-peer — the subagents should negotiate directly.
- B. ✓ Sequential — the second step needs the first step's output.
- C. Parallel — the subtasks are independent.
- D. Sequential — a later step depends on an earlier step's result.

Correct answer: B. Independent subtasks with no shared dependency are strong candidates for parallel execution; a subtask that genuinely needs a prior step's output must run sequentially after it.

Q28. Five independent log files are each scanned for anomalies by separate subagents. How should these subtasks be executed?

- A. ✓ Parallel — no subagent depends on another's output.
- B. Parallel — the subtasks are independent.
- C. Sequential — a later step depends on an earlier step's result.
- D. Peer-to-peer — the subagents should negotiate directly.

Correct answer: A. Independent subtasks with no shared dependency are strong candidates for parallel execution; a subtask that genuinely needs a prior step's output must run sequentially after it.

Q29. A draft is produced by one subagent, then reviewed by a second, independent subagent. How should these subtasks be executed?

- A. Parallel — the subtasks are independent.
- B. Sequential — a later step depends on an earlier step's result.
- C. Peer-to-peer — the subagents should negotiate directly.
- D. ✓ Sequential — review requires the draft to exist first.

Correct answer: D. Independent subtasks with no shared dependency are strong candidates for parallel execution; a subtask that genuinely needs a prior step's output must run sequentially after it.

Q30. Three subagents each call a different city's weather API with no shared state. How should these subtasks be executed?

- A. Sequential — a later step depends on an earlier step's result.
- B. Peer-to-peer — the subagents should negotiate directly.
- C. ✓ Parallel — fully independent calls.
- D. Parallel — the subtasks are independent.

Correct answer: C. Independent subtasks with no shared dependency are strong candidates for parallel execution; a subtask that genuinely needs a prior step's output must run sequentially after it.

Q31. Data must be extracted, then transformed, then loaded into a warehouse. How should these subtasks be executed?

- A. Peer-to-peer — the subagents should negotiate directly.
- B. ✓ Sequential — a classic pipeline with strict ordering.
- C. Parallel — the subtasks are independent.
- D. Sequential — a later step depends on an earlier step's result.

Correct answer: B. Independent subtasks with no shared dependency are strong candidates for parallel execution; a subtask that genuinely needs a prior step's output must run sequentially after it.

Q32. Four subagents each summarize a different, unrelated news article. How should these subtasks be executed?

- A. ✓ Parallel — the summaries don't depend on each other.
- B. Parallel — the subtasks are independent.
- C. Sequential — a later step depends on an earlier step's result.
- D. Peer-to-peer — the subagents should negotiate directly.

Correct answer: A. Independent subtasks with no shared dependency are strong candidates for parallel execution; a subtask that genuinely needs a prior step's output must run sequentially after it.

Q33. Mid-execution, a subagent returns an unexpected null result partway through the task. What should the coordinator do?

- A. Continue executing the original fixed plan unchanged.
- B. Abort the entire task immediately with no partial output.
- C. Ignore the new information and let a downstream step handle it silently.
- D. ✓ Revise the plan based on the new information — dynamic replanning, not blind continuation.

Correct answer: D. Good orchestration treats the plan as a living object: when intermediate results change what's known, the coordinator should re-evaluate and adjust rather than executing blindly to a stale plan.

Q34. Mid-execution, new information emerges mid-research that changes the relevant scope. What should the coordinator do?

- A. Abort the entire task immediately with no partial output.
- B. Ignore the new information and let a downstream step handle it silently.
- C. ✓ Revise the plan based on the new information — dynamic replanning, not blind continuation.
- D. Continue executing the original fixed plan unchanged.

Correct answer: C. Good orchestration treats the plan as a living object: when intermediate results change what's known, the coordinator should re-evaluate and adjust rather than executing blindly to a stale plan.

Q35. Mid-execution, a tool call reveals the task is substantially larger than originally estimated. What should the coordinator do?

- A. Ignore the new information and let a downstream step handle it silently.
- B. ✓ Revise the plan based on the new information — dynamic replanning, not blind continuation.
- C. Continue executing the original fixed plan unchanged.
- D. Abort the entire task immediately with no partial output.

Correct answer: B. Good orchestration treats the plan as a living object: when intermediate results change what's known, the coordinator should re-evaluate and adjust rather than executing blindly to a stale plan.

Q36. Mid-execution, the user changes a requirement while the task is already in progress. What should the coordinator do?

- A. ✓ Revise the plan based on the new information — dynamic replanning, not blind continuation.
- B. Continue executing the original fixed plan unchanged.
- C. Abort the entire task immediately with no partial output.
- D. Ignore the new information and let a downstream step handle it silently.

Correct answer: A. Good orchestration treats the plan as a living object: when intermediate results change what's known, the coordinator should re-evaluate and adjust rather than executing blindly to a stale plan.

Q37. Mid-execution, a subtask has failed twice after reasonable retries. What should the coordinator do?

- A. Continue executing the original fixed plan unchanged.
- B. Abort the entire task immediately with no partial output.
- C. Ignore the new information and let a downstream step handle it silently.
- D. ✓ Revise the plan based on the new information — dynamic replanning, not blind continuation.

Correct answer: D. Good orchestration treats the plan as a living object: when intermediate results change what's known, the coordinator should re-evaluate and adjust rather than executing blindly to a stale plan.

Q38. Mid-execution, a data source the plan depended on turns out to be deprecated. What should the coordinator do?

- A. Abort the entire task immediately with no partial output.
- B. Ignore the new information and let a downstream step handle it silently.
- C. ✓ Revise the plan based on the new information — dynamic replanning, not blind continuation.
- D. Continue executing the original fixed plan unchanged.

Correct answer: C. Good orchestration treats the plan as a living object: when intermediate results change what's known, the coordinator should re-evaluate and adjust rather than executing blindly to a stale plan.

Q39. Mid-execution, the task is approaching its allocated budget or token ceiling. What should the coordinator do?

- A. Ignore the new information and let a downstream step handle it silently.
- B. ✓ Revise the plan based on the new information — dynamic replanning, not blind continuation.
- C. Continue executing the original fixed plan unchanged.
- D. Abort the entire task immediately with no partial output.

Correct answer: B. Good orchestration treats the plan as a living object: when intermediate results change what's known, the coordinator should re-evaluate and adjust rather than executing blindly to a stale plan.

Q40. Mid-execution, two subagents return results that directly conflict with each other. What should the coordinator do?

- A. ✓ Revise the plan based on the new information — dynamic replanning, not blind continuation.
- B. Continue executing the original fixed plan unchanged.
- C. Abort the entire task immediately with no partial output.
- D. Ignore the new information and let a downstream step handle it silently.

Correct answer: A. Good orchestration treats the plan as a living object: when intermediate results change what's known, the coordinator should re-evaluate and adjust rather than executing blindly to a stale plan.

Q41. A fixed sequence of steps — parse input, call one classification model, format output — with no runtime branching. How should this system be classified?

- A. Agent
- B. Conversational system
- C. Single-shot prompt with no loop
- D. ✓ Workflow

Correct answer: D. Classification hinges on runtime autonomy: fixed steps with no model-driven branching is a workflow; human-driven turn-by-turn exchange is conversational; model-directed tool use and adaptive planning is agentic.

Q42. A system where the model decides which tools to call, in what order, and when the task is done, adapting as it goes. How should this system be classified?

- A. Conversational system
- B. Single-shot prompt with no loop
- C. ✓ Agent
- D. Workflow

Correct answer: C. Classification hinges on runtime autonomy: fixed steps with no model-driven branching is a workflow; human-driven turn-by-turn exchange is conversational; model-directed tool use and adaptive planning is agentic.

Q43. A chat interface where a human drives every turn and the model never acts without being prompted each time. How should this system be classified?

- A. Single-shot prompt with no loop
- B. ✓ Conversational system
- C. Agent
- D. Workflow

Correct answer: B. Classification hinges on runtime autonomy: fixed steps with no model-driven branching is a workflow; human-driven turn-by-turn exchange is conversational; model-directed tool use and adaptive planning is agentic.

Q44. A nightly ETL job that always runs the same three transformation steps regardless of the data. How should this system be classified?

- A. ✓ Workflow
- B. Agent
- C. Conversational system
- D. Single-shot prompt with no loop

Correct answer: A. Classification hinges on runtime autonomy: fixed steps with no model-driven branching is a workflow; human-driven turn-by-turn exchange is conversational; model-directed tool use and adaptive planning is agentic.

Q45. A research assistant that decides on its own how many sources to check and when it has enough to answer. How should this system be classified?

- A. Workflow
- B. Conversational system
- C. Single-shot prompt with no loop
- D. ✓ Agent

Correct answer: D. Classification hinges on runtime autonomy: fixed steps with no model-driven branching is a workflow; human-driven turn-by-turn exchange is conversational; model-directed tool use and adaptive planning is agentic.

Q46. A support bot that only ever responds to what the user just typed, never initiating its own actions. How should this system be classified?

- A. Workflow
- B. Single-shot prompt with no loop
- C. ✓ Conversational system
- D. Agent

Correct answer: C. Classification hinges on runtime autonomy: fixed steps with no model-driven branching is a workflow; human-driven turn-by-turn exchange is conversational; model-directed tool use and adaptive planning is agentic.

Q47. A system that dynamically decides to spin up additional subagents when a subtask proves more complex than expected. How should this system be classified?

- A. Single-shot prompt with no loop
- B. ✓ Agent
- C. Workflow
- D. Conversational system

Correct answer: B. Classification hinges on runtime autonomy: fixed steps with no model-driven branching is a workflow; human-driven turn-by-turn exchange is conversational; model-directed tool use and adaptive planning is agentic.

Q48. A fixed onboarding checklist bot that always asks the same five questions in the same order. How should this system be classified?

- A. ✓ Workflow
- B. Agent
- C. Conversational system
- D. Single-shot prompt with no loop

Correct answer: A. Classification hinges on runtime autonomy: fixed steps with no model-driven branching is a workflow; human-driven turn-by-turn exchange is conversational; model-directed tool use and adaptive planning is agentic.

Q49. A fully-specified ETL pipeline with one LLM classification step in the middle. Is agentic architecture (versus a simpler fixed workflow or single prompt) justified here?

- A. Yes — flip the framing regardless of the scenario's actual structure.
- B. Yes, agentic architecture is always the safer default choice.
- C. No, because LLMs cannot be trusted with any autonomy.
- D. ✓ No — the steps are fully knowable ahead of time.

Correct answer: D. Agentic architecture earns its added latency, cost, and failure surface when scope is open-ended or the plan must adapt at runtime; fully-knowable, fixed sequences are better served by a simpler workflow.

Q50. An open-ended competitive research task where the right sources aren't known in advance. Is agentic architecture (versus a simpler fixed workflow or single prompt) justified here?

- A. Yes, agentic architecture is always the safer default choice.
- B. No, because LLMs cannot be trusted with any autonomy.
- C. ✓ Yes — scope is open-ended and requires adaptive planning.
- D. No — flip the framing regardless of the scenario's actual structure.

Correct answer: C. Agentic architecture earns its added latency, cost, and failure surface when scope is open-ended or the plan must adapt at runtime; fully-knowable, fixed sequences are better served by a simpler workflow.

Q51. A fixed weekly report that always pulls the same three metrics in the same format. Is agentic architecture (versus a simpler fixed workflow or single prompt) justified here?

- A. No, because LLMs cannot be trusted with any autonomy.
- B. ✓ No — the sequence never varies and needs no runtime judgment.
- C. Yes — flip the framing regardless of the scenario's actual structure.
- D. Yes, agentic architecture is always the safer default choice.

Correct answer: B. Agentic architecture earns its added latency, cost, and failure surface when scope is open-ended or the plan must adapt at runtime; fully-knowable, fixed sequences are better served by a simpler workflow.

Q52. A customer-support task that may require zero, one, or several tool calls depending on the issue. Is agentic architecture (versus a simpler fixed workflow or single prompt) justified here?

- A. ✓ Yes — the number and order of tool calls can't be fixed in advance.
- B. No — flip the framing regardless of the scenario's actual structure.
- C. Yes, agentic architecture is always the safer default choice.
- D. No, because LLMs cannot be trusted with any autonomy.

Correct answer: A. Agentic architecture earns its added latency, cost, and failure surface when scope is open-ended or the plan must adapt at runtime; fully-knowable, fixed sequences are better served by a simpler workflow.

Q53. A single well-structured prompt that reliably classifies a support ticket into one of five categories. Is agentic architecture (versus a simpler fixed workflow or single prompt) justified here?

- A. Yes — flip the framing regardless of the scenario's actual structure.
- B. Yes, agentic architecture is always the safer default choice.
- C. No, because LLMs cannot be trusted with any autonomy.
- D. ✓ No — a simple prompt already solves this without added orchestration.

Correct answer: D. Agentic architecture earns its added latency, cost, and failure surface when scope is open-ended or the plan must adapt at runtime; fully-knowable, fixed sequences are better served by a simpler workflow.

Q54. A multi-step investigation where each finding determines what to look into next. Is agentic architecture (versus a simpler fixed workflow or single prompt) justified here?

- A. Yes, agentic architecture is always the safer default choice.
- B. No, because LLMs cannot be trusted with any autonomy.
- C. ✓ Yes — later steps genuinely depend on adapting to earlier findings.
- D. No — flip the framing regardless of the scenario's actual structure.

Correct answer: C. Agentic architecture earns its added latency, cost, and failure surface when scope is open-ended or the plan must adapt at runtime; fully-knowable, fixed sequences are better served by a simpler workflow.

Q55. A nightly batch job with a rigid, unchanging list of files to process in order. Is agentic architecture (versus a simpler fixed workflow or single prompt) justified here?

- A. No, because LLMs cannot be trusted with any autonomy.
- B. ✓ No — there is no need for model-driven autonomy over the sequence.
- C. Yes — flip the framing regardless of the scenario's actual structure.
- D. Yes, agentic architecture is always the safer default choice.

Correct answer: B. Agentic architecture earns its added latency, cost, and failure surface when scope is open-ended or the plan must adapt at runtime; fully-knowable, fixed sequences are better served by a simpler workflow.

Q56. A research task where the number of necessary subagents can't be known until initial findings come in. Is agentic architecture (versus a simpler fixed workflow or single prompt) justified here?

- A. ✓ Yes — the plan must adapt to information only available mid-task.
- B. No — flip the framing regardless of the scenario's actual structure.
- C. Yes, agentic architecture is always the safer default choice.
- D. No, because LLMs cannot be trusted with any autonomy.

Correct answer: A. Agentic architecture earns its added latency, cost, and failure surface when scope is open-ended or the plan must adapt at runtime; fully-knowable, fixed sequences are better served by a simpler workflow.

Q57. In a hub-and-spoke multi-agent system, who owns: "Task decomposition and scoping"?

- A. Subagent
- B. Both equally
- C. Neither — this belongs to a human
- D. ✓ Coordinator

Correct answer: D. The coordinator owns decomposition, delegation, error handling, escalation, and synthesis; subagents own only the narrow execution of what they were explicitly assigned.

Q58. In a hub-and-spoke multi-agent system, who owns: "Packing relevant context into each subagent's task prompt"?

- A. Both equally
- B. Neither — this belongs to a human
- C. ✓ Coordinator
- D. Subagent

Correct answer: C. The coordinator owns decomposition, delegation, error handling, escalation, and synthesis; subagents own only the narrow execution of what they were explicitly assigned.

Q59. In a hub-and-spoke multi-agent system, who owns: "Executing one narrow, well-scoped subtask"?

- A. Neither — this belongs to a human
- B. ✓ Subagent
- C. Coordinator
- D. Both equally

Correct answer: B. The coordinator owns decomposition, delegation, error handling, escalation, and synthesis; subagents own only the narrow execution of what they were explicitly assigned.

Q60. In a hub-and-spoke multi-agent system, who owns: "Cross-subagent error handling and retry policy"?

- A. ✓ Coordinator
- B. Subagent
- C. Both equally
- D. Neither — this belongs to a human

Correct answer: A. The coordinator owns decomposition, delegation, error handling, escalation, and synthesis; subagents own only the narrow execution of what they were explicitly assigned.

Q61. In a hub-and-spoke multi-agent system, who owns: "Final result aggregation and synthesis"?

- A. Subagent
- B. Both equally
- C. Neither — this belongs to a human
- D. ✓ Coordinator

Correct answer: D. The coordinator owns decomposition, delegation, error handling, escalation, and synthesis; subagents own only the narrow execution of what they were explicitly assigned.

Q62. In a hub-and-spoke multi-agent system, who owns: "Deciding whether to escalate to a human"?

- A. Both equally
- B. Neither — this belongs to a human
- C. ✓ Coordinator
- D. Subagent

Correct answer: C. The coordinator owns decomposition, delegation, error handling, escalation, and synthesis; subagents own only the narrow execution of what they were explicitly assigned.

Q63. In a hub-and-spoke multi-agent system, who owns: "Delegating a subtask to the right subagent"?

- A. Neither — this belongs to a human
- B. ✓ Coordinator
- C. Subagent
- D. Both equally

Correct answer: B. The coordinator owns decomposition, delegation, error handling, escalation, and synthesis; subagents own only the narrow execution of what they were explicitly assigned.

Q64. In a hub-and-spoke multi-agent system, who owns: "Defining the exact scope boundaries of a subtask"?

- A. ✓ Coordinator
- B. Subagent
- C. Both equally
- D. Neither — this belongs to a human

Correct answer: A. The coordinator owns decomposition, delegation, error handling, escalation, and synthesis; subagents own only the narrow execution of what they were explicitly assigned.

Q65. In a hub-and-spoke multi-agent system, who owns: "Reporting its own result back without inventing scope beyond its assignment"?

- A. Coordinator
- B. Both equally
- C. Neither — this belongs to a human
- D. ✓ Subagent

Correct answer: D. The coordinator owns decomposition, delegation, error handling, escalation, and synthesis; subagents own only the narrow execution of what they were explicitly assigned.

Q66. In a hub-and-spoke multi-agent system, who owns: "Reconciling conflicting results returned by two subagents"?

- A. Both equally
- B. Neither — this belongs to a human
- C. ✓ Coordinator
- D. Subagent

Correct answer: C. The coordinator owns decomposition, delegation, error handling, escalation, and synthesis; subagents own only the narrow execution of what they were explicitly assigned.

Q67. Which of the following is the LEAST reliable trigger for escalating an agent interaction to a human?

- A. demonstrated inability to make progress after reasonable attempts
- B. ✓ a sentiment classifier flagging the conversation as negative
- C. an explicit user request for a human
- D. a genuine policy gap the agent has no authority to interpret

Correct answer: B. Sentiment analysis, self-reported model confidence, and soft classifier scores are all internally-generated or statistical signals that can be miscalibrated; explicit requests and structurally verifiable conditions are the reliable category.

Q68. Which of the following is the LEAST reliable trigger for escalating an agent interaction to a human?

- A. ✓ the model's self-reported confidence score
- B. an explicit user request for a human
- C. a genuine policy gap the agent has no authority to interpret
- D. an automatic soft-label classifier's complexity score

Correct answer: A. Sentiment analysis, self-reported model confidence, and soft classifier scores are all internally-generated or statistical signals that can be miscalibrated; explicit requests and structurally verifiable conditions are the reliable category.

Q69. Which of the following is the LEAST reliable trigger for escalating an agent interaction to a human?

- A. a genuine policy gap the agent has no authority to interpret
- B. demonstrated inability to make progress after reasonable attempts
- C. the model's self-reported confidence score
- D. ✓ a sentiment classifier flagging the conversation as negative

Correct answer: D. Sentiment analysis, self-reported model confidence, and soft classifier scores are all internally-generated or statistical signals that can be miscalibrated; explicit requests and structurally verifiable conditions are the reliable category.

Q70. Which of the following is the LEAST reliable trigger for escalating an agent interaction to a human?

- A. the model's self-reported confidence score
- B. an automatic soft-label classifier's complexity score
- C. ✓ a sentiment classifier flagging the conversation as negative
- D. an explicit user request for a human

Correct answer: C. Sentiment analysis, self-reported model confidence, and soft classifier scores are all internally-generated or statistical signals that can be miscalibrated; explicit requests and structurally verifiable conditions are the reliable category.

Q71. Which of the following is the LEAST reliable trigger for escalating an agent interaction to a human?

- A. a sentiment classifier flagging the conversation as negative
- B. ✓ an automatic soft-label classifier's complexity score
- C. demonstrated inability to make progress after reasonable attempts
- D. an explicit user request for a human

Correct answer: B. Sentiment analysis, self-reported model confidence, and soft classifier scores are all internally-generated or statistical signals that can be miscalibrated; explicit requests and structurally verifiable conditions are the reliable category.

Q72. Which of the following is the LEAST reliable trigger for escalating an agent interaction to a human?

- A. ✓ the model's self-reported confidence score
- B. a genuine policy gap the agent has no authority to interpret
- C. an explicit user request for a human
- D. an automatic soft-label classifier's complexity score

Correct answer: A. Sentiment analysis, self-reported model confidence, and soft classifier scores are all internally-generated or statistical signals that can be miscalibrated; explicit requests and structurally verifiable conditions are the reliable category.

Q73. Which of the following is the LEAST reliable trigger for escalating an agent interaction to a human?

- A. demonstrated inability to make progress after reasonable attempts
- B. the model's self-reported confidence score
- C. an explicit user request for a human
- D. ✓ a sentiment classifier flagging the conversation as negative

Correct answer: D. Sentiment analysis, self-reported model confidence, and soft classifier scores are all internally-generated or statistical signals that can be miscalibrated; explicit requests and structurally verifiable conditions are the reliable category.

Q74. Which of the following is the LEAST reliable trigger for escalating an agent interaction to a human?

- A. an automatic soft-label classifier's complexity score
- B. a genuine policy gap the agent has no authority to interpret
- C. ✓ a sentiment classifier flagging the conversation as negative
- D. the model's self-reported confidence score

Correct answer: C. Sentiment analysis, self-reported model confidence, and soft classifier scores are all internally-generated or statistical signals that can be miscalibrated; explicit requests and structurally verifiable conditions are the reliable category.

Q75. A subagent produces a plan that exceeds the client's stated budget, even though the coordinator seemingly "knew" that constraint. Why?

- A. The synthesis step corrupted a correct subagent result.
- B. ✓ The coordinator never explicitly passed a budget constraint into the subagent's task prompt.
- C. The subagent ignored instructions it was actually given.
- D. The model's context window was too small to hold the task.

Correct answer: B. Subagents run in isolated context and inherit nothing automatically from the coordinator; any constraint the coordinator doesn't explicitly restate in the task prompt effectively does not exist for that subagent.

Q76. A subagent duplicates research another subagent already completed, even though the coordinator seemingly "knew" that constraint. Why?

- A. ✓ The coordinator never explicitly passed prior findings from an earlier subagent into the subagent's task prompt.
- B. The subagent ignored instructions it was actually given.
- C. The model's context window was too small to hold the task.
- D. The synthesis step corrupted a correct subagent result.

Correct answer: A. Subagents run in isolated context and inherit nothing automatically from the coordinator; any constraint the coordinator doesn't explicitly restate in the task prompt effectively does not exist for that subagent.

Q77. A subagent returns free text instead of the structured format the coordinator needed, even though the coordinator seemingly "knew" that constraint. Why?

- A. The subagent ignored instructions it was actually given.
- B. The model's context window was too small to hold the task.
- C. The synthesis step corrupted a correct subagent result.
- D. ✓ The coordinator never explicitly passed the required output format into the subagent's task prompt.

Correct answer: D. Subagents run in isolated context and inherit nothing automatically from the coordinator; any constraint the coordinator doesn't explicitly restate in the task prompt effectively does not exist for that subagent.

Q78. A subagent produces a timeline that ignores the actual due date, even though the coordinator seemingly "knew" that constraint. Why?

- A. The model's context window was too small to hold the task.
- B. The synthesis step corrupted a correct subagent result.
- C. ✓ The coordinator never explicitly passed a hard deadline into the subagent's task prompt.
- D. The subagent ignored instructions it was actually given.

Correct answer: C. Subagents run in isolated context and inherit nothing automatically from the coordinator; any constraint the coordinator doesn't explicitly restate in the task prompt effectively does not exist for that subagent.

Q79. A subagent writes in a tone that doesn't match who the report is for, even though the coordinator seemingly "knew" that constraint. Why?

- A. The synthesis step corrupted a correct subagent result.
- B. ✓ The coordinator never explicitly passed the target audience into the subagent's task prompt.
- C. The subagent ignored instructions it was actually given.
- D. The model's context window was too small to hold the task.

Correct answer: B. Subagents run in isolated context and inherit nothing automatically from the coordinator; any constraint the coordinator doesn't explicitly restate in the task prompt effectively does not exist for that subagent.

Q80. A subagent repeats the exact approach that already failed once, even though the coordinator seemingly "knew" that constraint. Why?

- A. ✓ The coordinator never explicitly passed context about a prior failed attempt into the subagent's task prompt.
- B. The subagent ignored instructions it was actually given.
- C. The model's context window was too small to hold the task.
- D. The synthesis step corrupted a correct subagent result.

Correct answer: A. Subagents run in isolated context and inherit nothing automatically from the coordinator; any constraint the coordinator doesn't explicitly restate in the task prompt effectively does not exist for that subagent.

Q81. A subagent answers a narrower question than what was actually asked, even though the coordinator seemingly "knew" that constraint. Why?

- A. The subagent ignored instructions it was actually given.
- B. The model's context window was too small to hold the task.
- C. The synthesis step corrupted a correct subagent result.
- D. ✓ The coordinator never explicitly passed the user's original underlying question into the subagent's task prompt.

Correct answer: D. Subagents run in isolated context and inherit nothing automatically from the coordinator; any constraint the coordinator doesn't explicitly restate in the task prompt effectively does not exist for that subagent.

Q82. A subagent produces output that violates house style rules, even though the coordinator seemingly "knew" that constraint. Why?

- A. The model's context window was too small to hold the task.
- B. The synthesis step corrupted a correct subagent result.
- C. ✓ The coordinator never explicitly passed the required style guide into the subagent's task prompt.
- D. The subagent ignored instructions it was actually given.

Correct answer: C. Subagents run in isolated context and inherit nothing automatically from the coordinator; any constraint the coordinator doesn't explicitly restate in the task prompt effectively does not exist for that subagent.

Q83. A central agent needs to decompose one broad task into independent pieces and merge the results. Which multi-agent topology best fits?

- A. Fully decentralized swarm with no defined roles
- B. ✓ Hub-and-spoke
- C. Pipeline
- D. Peer-to-peer

Correct answer: B. Hub-and-spoke fits central decomposition/aggregation, pipeline fits strict sequential handoff, and peer-to-peer fits decentralized negotiation with no single owning coordinator.

Q84. Data must flow through extract, validate, and format stages in strict, unchanging order. Which multi-agent topology best fits?

- A. ✓ Pipeline
- B. Hub-and-spoke
- C. Peer-to-peer
- D. Fully decentralized swarm with no defined roles

Correct answer: A. Hub-and-spoke fits central decomposition/aggregation, pipeline fits strict sequential handoff, and peer-to-peer fits decentralized negotiation with no single owning coordinator.

Q85. Several agents need to negotiate directly with each other with no single owner of the outcome. Which multi-agent topology best fits?

- A. Hub-and-spoke
- B. Pipeline
- C. Fully decentralized swarm with no defined roles
- D. ✓ Peer-to-peer

Correct answer: D. Hub-and-spoke fits central decomposition/aggregation, pipeline fits strict sequential handoff, and peer-to-peer fits decentralized negotiation with no single owning coordinator.

Q86. A researcher agent, a fact-checker agent, and a writer agent must each build on the previous stage's output. Which multi-agent topology best fits?

- A. Peer-to-peer
- B. Fully decentralized swarm with no defined roles
- C. ✓ Pipeline
- D. Hub-and-spoke

Correct answer: C. Hub-and-spoke fits central decomposition/aggregation, pipeline fits strict sequential handoff, and peer-to-peer fits decentralized negotiation with no single owning coordinator.

Q87. A single coordinator manages five independent research streams and combines them into one report. Which multi-agent topology best fits?

- A. Fully decentralized swarm with no defined roles
- B. ✓ Hub-and-spoke
- C. Pipeline
- D. Peer-to-peer

Correct answer: B. Hub-and-spoke fits central decomposition/aggregation, pipeline fits strict sequential handoff, and peer-to-peer fits decentralized negotiation with no single owning coordinator.

Q88. Two specialized agents must jointly negotiate a shared plan with no coordinator arbitrating. Which multi-agent topology best fits?

- A. ✓ Peer-to-peer
- B. Hub-and-spoke
- C. Pipeline
- D. Fully decentralized swarm with no defined roles

Correct answer: A. Hub-and-spoke fits central decomposition/aggregation, pipeline fits strict sequential handoff, and peer-to-peer fits decentralized negotiation with no single owning coordinator.

Q89. A support system routes each ticket to exactly one of several independent specialist subagents. Which multi-agent topology best fits?

- A. Pipeline
- B. Peer-to-peer
- C. Fully decentralized swarm with no defined roles
- D. ✓ Hub-and-spoke

Correct answer: D. Hub-and-spoke fits central decomposition/aggregation, pipeline fits strict sequential handoff, and peer-to-peer fits decentralized negotiation with no single owning coordinator.

Q90. A document must pass sequentially through drafting, legal review, and final formatting. Which multi-agent topology best fits?

- A. Peer-to-peer
- B. Fully decentralized swarm with no defined roles
- C. ✓ Pipeline
- D. Hub-and-spoke

Correct answer: C. Hub-and-spoke fits central decomposition/aggregation, pipeline fits strict sequential handoff, and peer-to-peer fits decentralized negotiation with no single owning coordinator.

Q91. A coordinator caps subagent loops at 10 iterations as a safety ceiling. What should happen when that cap is hit?

- A. Increase the cap indefinitely until the loop naturally stops.
- B. ✓ Treat the hit ceiling as a distinct failure mode to surface, not a silent success.
- C. Treat reaching the cap as successful task completion.
- D. Silently return whatever partial output exists with no flag.

Correct answer: B. A hit iteration cap is a safety ceiling, not a completion signal — it should trigger explicit handling, not be silently treated as if the task finished normally.

Q92. A coordinator caps subagent loops at 15 iterations as a safety ceiling. What should happen when that cap is hit?

- A. ✓ Treat the hit ceiling as a distinct failure mode to surface, not a silent success.
- B. Treat reaching the cap as successful task completion.
- C. Silently return whatever partial output exists with no flag.
- D. Increase the cap indefinitely until the loop naturally stops.

Correct answer: A. A hit iteration cap is a safety ceiling, not a completion signal — it should trigger explicit handling, not be silently treated as if the task finished normally.

Q93. A coordinator caps subagent loops at 20 iterations as a safety ceiling. What should happen when that cap is hit?

- A. Treat reaching the cap as successful task completion.
- B. Silently return whatever partial output exists with no flag.
- C. Increase the cap indefinitely until the loop naturally stops.
- D. ✓ Treat the hit ceiling as a distinct failure mode to surface, not a silent success.

Correct answer: D. A hit iteration cap is a safety ceiling, not a completion signal — it should trigger explicit handling, not be silently treated as if the task finished normally.

Q94. A coordinator caps subagent loops at 25 iterations as a safety ceiling. What should happen when that cap is hit?

- A. Silently return whatever partial output exists with no flag.
- B. Increase the cap indefinitely until the loop naturally stops.
- C. ✓ Treat the hit ceiling as a distinct failure mode to surface, not a silent success.
- D. Treat reaching the cap as successful task completion.

Correct answer: C. A hit iteration cap is a safety ceiling, not a completion signal — it should trigger explicit handling, not be silently treated as if the task finished normally.

Q95. A ticket classification task that used to run as a single prompt now runs as a five-subagent agentic pipeline, tripling cost and latency with no accuracy gain. What's the right response?

- A. Switch to a larger model to reduce the number of steps needed.
- B. ✓ Reconsider whether a simpler workflow or single prompt would achieve the same result more cheaply and reliably.
- C. Add more subagents to parallelize the cost away.
- D. Ignore cost — agentic systems should always maximize autonomy.

Correct answer: B. Agentic architecture isn't free — added autonomy should be justified by open-ended scope or adaptive planning needs; if a simpler pattern gets the same result, added agent complexity is the wrong tradeoff.

Q96. A weekly summary task that used to run as a single prompt now runs as a five-subagent agentic pipeline, tripling cost and latency with no accuracy gain. What's the right response?

- A. ✓ Reconsider whether a simpler workflow or single prompt would achieve the same result more cheaply and reliably.
- B. Add more subagents to parallelize the cost away.
- C. Ignore cost — agentic systems should always maximize autonomy.
- D. Switch to a larger model to reduce the number of steps needed.

Correct answer: A. Agentic architecture isn't free — added autonomy should be justified by open-ended scope or adaptive planning needs; if a simpler pattern gets the same result, added agent complexity is the wrong tradeoff.

Q97. A invoice-field extraction task that used to run as a single prompt now runs as a five-subagent agentic pipeline, tripling cost and latency with no accuracy gain. What's the right response?

- A. Add more subagents to parallelize the cost away.
- B. Ignore cost — agentic systems should always maximize autonomy.
- C. Switch to a larger model to reduce the number of steps needed.
- D. ✓ Reconsider whether a simpler workflow or single prompt would achieve the same result more cheaply and reliably.

Correct answer: D. Agentic architecture isn't free — added autonomy should be justified by open-ended scope or adaptive planning needs; if a simpler pattern gets the same result, added agent complexity is the wrong tradeoff.

Q98. A FAQ-answering task that used to run as a single prompt now runs as a five-subagent agentic pipeline, tripling cost and latency with no accuracy gain. What's the right response?

- A. Ignore cost — agentic systems should always maximize autonomy.
- B. Switch to a larger model to reduce the number of steps needed.
- C. ✓ Reconsider whether a simpler workflow or single prompt would achieve the same result more cheaply and reliably.
- D. Add more subagents to parallelize the cost away.

Correct answer: C. Agentic architecture isn't free — added autonomy should be justified by open-ended scope or adaptive planning needs; if a simpler pattern gets the same result, added agent complexity is the wrong tradeoff.

Q99. A subagent at nesting level 2 spawns its own subagents, which then spawn further subagents, and cost balloons unexpectedly. What design control should have been in place?

- A. Let each subagent decide its own depth limit independently.
- B. ✓ Cap recursion depth explicitly and require a clear justification before a subagent may spawn its own subagents.
- C. Allow unlimited recursive delegation with no depth limit.
- D. Disallow all subagent-of-subagent patterns in every case.

Correct answer: B. Deep or unbounded recursive delegation can silently multiply cost and latency; a deliberate, explicit depth cap with justification keeps nested delegation from spiraling.

Q100. A subagent at nesting level 3 spawns its own subagents, which then spawn further subagents, and cost balloons unexpectedly. What design control should have been in place?

- A. ✓ Cap recursion depth explicitly and require a clear justification before a subagent may spawn its own subagents.
- B. Allow unlimited recursive delegation with no depth limit.
- C. Disallow all subagent-of-subagent patterns in every case.
- D. Let each subagent decide its own depth limit independently.

Correct answer: A. Deep or unbounded recursive delegation can silently multiply cost and latency; a deliberate, explicit depth cap with justification keeps nested delegation from spiraling.

Q101. A subagent at nesting level 4 spawns its own subagents, which then spawn further subagents, and cost balloons unexpectedly. What design control should have been in place?

- A. Allow unlimited recursive delegation with no depth limit.
- B. Disallow all subagent-of-subagent patterns in every case.
- C. Let each subagent decide its own depth limit independently.
- D. ✓ Cap recursion depth explicitly and require a clear justification before a subagent may spawn its own subagents.

Correct answer: D. Deep or unbounded recursive delegation can silently multiply cost and latency; a deliberate, explicit depth cap with justification keeps nested delegation from spiraling.

Q102. A subagent at nesting level 5 spawns its own subagents, which then spawn further subagents, and cost balloons unexpectedly. What design control should have been in place?

- A. Disallow all subagent-of-subagent patterns in every case.
- B. Let each subagent decide its own depth limit independently.
- C. ✓ Cap recursion depth explicitly and require a clear justification before a subagent may spawn its own subagents.
- D. Allow unlimited recursive delegation with no depth limit.

Correct answer: C. Deep or unbounded recursive delegation can silently multiply cost and latency; a deliberate, explicit depth cap with justification keeps nested delegation from spiraling.

Q103. In a batch of tool calls, a lookup call and a calculation call that consumes its result must run in order because the second needs the first's output, while the rest are independent. How should the orchestration layer handle this?

- A. Let the model guess the order with no explicit sequencing logic.
- B. ✓ Detect the dependency and force those specific calls to run in the required order, even if the rest run in parallel.
- C. Run every tool call in parallel regardless of dependency.
- D. Run every tool call strictly sequentially regardless of independence.

Correct answer: B. Mixed dependency graphs need selective sequencing: force genuinely dependent calls into order while still parallelizing everything that's actually independent.

Q104. In a batch of tool calls, an authentication call and the API call that requires its token must run in order because the second needs the first's output, while the rest are independent. How should the orchestration layer handle this?

- A. ✓ Detect the dependency and force those specific calls to run in the required order, even if the rest run in parallel.
- B. Run every tool call in parallel regardless of dependency.
- C. Run every tool call strictly sequentially regardless of independence.
- D. Let the model guess the order with no explicit sequencing logic.

Correct answer: A. Mixed dependency graphs need selective sequencing: force genuinely dependent calls into order while still parallelizing everything that's actually independent.

Q105. In a batch of tool calls, a file-creation call and a subsequent file-read call must run in order because the second needs the first's output, while the rest are independent. How should the orchestration layer handle this?

- A. Run every tool call in parallel regardless of dependency.
- B. Run every tool call strictly sequentially regardless of independence.
- C. Let the model guess the order with no explicit sequencing logic.
- D. ✓ Detect the dependency and force those specific calls to run in the required order, even if the rest run in parallel.

Correct answer: D. Mixed dependency graphs need selective sequencing: force genuinely dependent calls into order while still parallelizing everything that's actually independent.

Q106. In a batch of tool calls, a database write and a downstream read of that same record must run in order because the second needs the first's output, while the rest are independent. How should the orchestration layer handle this?

- A. Run every tool call strictly sequentially regardless of independence.
- B. Let the model guess the order with no explicit sequencing logic.
- C. ✓ Detect the dependency and force those specific calls to run in the required order, even if the rest run in parallel.
- D. Run every tool call in parallel regardless of dependency.

Correct answer: C. Mixed dependency graphs need selective sequencing: force genuinely dependent calls into order while still parallelizing everything that's actually independent.

Q107. A subagent fails on a transient network timeout after one attempt. Who should own the decision of whether and how to retry?

- A. No one — a single subagent failure should always abort the whole task.
- B. ✓ The coordinator, since it owns cross-subagent error handling and retry policy.
- C. Each subagent independently, with no coordinator involvement.
- D. The end user, who should be asked to manually retry.

Correct answer: B. Retry policy across subagents is a coordinator responsibility — it's the coordinator that has visibility into overall task state needed to decide whether retrying, falling back, or escalating makes sense.

Q108. A subagent fails on a rate-limited API call after one attempt. Who should own the decision of whether and how to retry?

- A. ✓ The coordinator, since it owns cross-subagent error handling and retry policy.
- B. Each subagent independently, with no coordinator involvement.
- C. The end user, who should be asked to manually retry.
- D. No one — a single subagent failure should always abort the whole task.

Correct answer: A. Retry policy across subagents is a coordinator responsibility — it's the coordinator that has visibility into overall task state needed to decide whether retrying, falling back, or escalating makes sense.

Q109. A subagent fails on an ambiguous tool result after one attempt. Who should own the decision of whether and how to retry?

- A. Each subagent independently, with no coordinator involvement.
- B. The end user, who should be asked to manually retry.
- C. No one — a single subagent failure should always abort the whole task.
- D. ✓ The coordinator, since it owns cross-subagent error handling and retry policy.

Correct answer: D. Retry policy across subagents is a coordinator responsibility — it's the coordinator that has visibility into overall task state needed to decide whether retrying, falling back, or escalating makes sense.

Q110. A subagent fails on a temporarily unavailable data source after one attempt. Who should own the decision of whether and how to retry?

- A. The end user, who should be asked to manually retry.
- B. No one — a single subagent failure should always abort the whole task.
- C. ✓ The coordinator, since it owns cross-subagent error handling and retry policy.
- D. Each subagent independently, with no coordinator involvement.

Correct answer: C. Retry policy across subagents is a coordinator responsibility — it's the coordinator that has visibility into overall task state needed to decide whether retrying, falling back, or escalating makes sense.

Q111. An agent that only needs to read customer order status is granted broad, unscoped account permissions "to be safe." What's the better practice?

- A. Grant broad permissions once and never revisit them.
- B. ✓ Scope the agent's tool permissions narrowly to only what that specific task requires.
- C. Grant the agent full account-wide permissions for convenience.
- D. Rely on the system prompt alone to describe what the agent shouldn't do.

Correct answer: B. Least-privilege scoping limits blast radius if the agent misfires or is manipulated — permissions should match the specific task, not be granted broadly by default.

Q112. An agent that only needs to post to a single internal Slack channel is granted broad, unscoped account permissions "to be safe." What's the better practice?

- A. ✓ Scope the agent's tool permissions narrowly to only what that specific task requires.
- B. Grant the agent full account-wide permissions for convenience.
- C. Rely on the system prompt alone to describe what the agent shouldn't do.
- D. Grant broad permissions once and never revisit them.

Correct answer: A. Least-privilege scoping limits blast radius if the agent misfires or is manipulated — permissions should match the specific task, not be granted broadly by default.

Q113. An agent that only needs to summarize log files is granted broad, unscoped account permissions "to be safe." What's the better practice?

- A. Grant the agent full account-wide permissions for convenience.
- B. Rely on the system prompt alone to describe what the agent shouldn't do.
- C. Grant broad permissions once and never revisit them.
- D. ✓ Scope the agent's tool permissions narrowly to only what that specific task requires.

Correct answer: D. Least-privilege scoping limits blast radius if the agent misfires or is manipulated — permissions should match the specific task, not be granted broadly by default.

Q114. An agent that only needs to look up inventory counts is granted broad, unscoped account permissions "to be safe." What's the better practice?

- A. Rely on the system prompt alone to describe what the agent shouldn't do.
- B. Grant broad permissions once and never revisit them.
- C. ✓ Scope the agent's tool permissions narrowly to only what that specific task requires.
- D. Grant the agent full account-wide permissions for convenience.

Correct answer: C. Least-privilege scoping limits blast radius if the agent misfires or is manipulated — permissions should match the specific task, not be granted broadly by default.

Q115. A third-party payment API call in an agent loop occasionally hangs with no response for several minutes. What should the orchestration layer do?

- A. Silently retry forever until the call eventually succeeds.
- B. ✓ Set an explicit timeout per tool call and treat a timeout as a distinct, handleable failure — not a silent hang.
- C. Let tool calls run with no timeout at all.
- D. Treat any slow call as a permanent failure with no retry option.

Correct answer: B. Explicit per-call timeouts turn an indefinite hang into a defined, handleable failure the coordinator can act on, rather than silently blocking the whole loop.

Q116. A internal search index call in an agent loop occasionally hangs with no response for several minutes. What should the orchestration layer do?

- A. ✓ Set an explicit timeout per tool call and treat a timeout as a distinct, handleable failure — not a silent hang.
- B. Let tool calls run with no timeout at all.
- C. Treat any slow call as a permanent failure with no retry option.
- D. Silently retry forever until the call eventually succeeds.

Correct answer: A. Explicit per-call timeouts turn an indefinite hang into a defined, handleable failure the coordinator can act on, rather than silently blocking the whole loop.

Q117. A external weather API call in an agent loop occasionally hangs with no response for several minutes. What should the orchestration layer do?

- A. Let tool calls run with no timeout at all.
- B. Treat any slow call as a permanent failure with no retry option.
- C. Silently retry forever until the call eventually succeeds.
- D. ✓ Set an explicit timeout per tool call and treat a timeout as a distinct, handleable failure — not a silent hang.

Correct answer: D. Explicit per-call timeouts turn an indefinite hang into a defined, handleable failure the coordinator can act on, rather than silently blocking the whole loop.

Q118. A legacy database query call in an agent loop occasionally hangs with no response for several minutes. What should the orchestration layer do?

- A. Treat any slow call as a permanent failure with no retry option.
- B. Silently retry forever until the call eventually succeeds.
- C. ✓ Set an explicit timeout per tool call and treat a timeout as a distinct, handleable failure — not a silent hang.
- D. Let tool calls run with no timeout at all.

Correct answer: C. Explicit per-call timeouts turn an indefinite hang into a defined, handleable failure the coordinator can act on, rather than silently blocking the whole loop.

Q119. A system automates fund transfers and includes a human approval step, but the step fires only in a summary email after the action has already run. What's wrong with this design?

- A. Place the checkpoint immediately after the action, as a post-hoc notification only.
- B. ✓ Place the checkpoint before the irreversible or high-stakes action, not after it has already executed.
- C. Place the checkpoint only at the very end of the entire task.
- D. Skip human checkpoints entirely if the agent seems confident.

Correct answer: B. A human-in-the-loop checkpoint only has value if it can prevent an irreversible or high-stakes action; placing it after execution turns it into a notification, not a safeguard.

Q120. A system automates production deployments and includes a human approval step, but the step fires only in a summary email after the action has already run. What's wrong with this design?

- A. ✓ Place the checkpoint before the irreversible or high-stakes action, not after it has already executed.
- B. Place the checkpoint only at the very end of the entire task.
- C. Skip human checkpoints entirely if the agent seems confident.
- D. Place the checkpoint immediately after the action, as a post-hoc notification only.

Correct answer: A. A human-in-the-loop checkpoint only has value if it can prevent an irreversible or high-stakes action; placing it after execution turns it into a notification, not a safeguard.

Q121. A system automates customer account closures and includes a human approval step, but the step fires only in a summary email after the action has already run. What's wrong with this design?

- A. Place the checkpoint only at the very end of the entire task.
- B. Skip human checkpoints entirely if the agent seems confident.
- C. Place the checkpoint immediately after the action, as a post-hoc notification only.
- D. ✓ Place the checkpoint before the irreversible or high-stakes action, not after it has already executed.

Correct answer: D. A human-in-the-loop checkpoint only has value if it can prevent an irreversible or high-stakes action; placing it after execution turns it into a notification, not a safeguard.

Q122. A system automates bulk data deletions and includes a human approval step, but the step fires only in a summary email after the action has already run. What's wrong with this design?

- A. Skip human checkpoints entirely if the agent seems confident.
- B. Place the checkpoint immediately after the action, as a post-hoc notification only.
- C. ✓ Place the checkpoint before the irreversible or high-stakes action, not after it has already executed.
- D. Place the checkpoint only at the very end of the entire task.

Correct answer: C. A human-in-the-loop checkpoint only has value if it can prevent an irreversible or high-stakes action; placing it after execution turns it into a notification, not a safeguard.

Q123. A multi-agent research pipeline occasionally produces an incorrect final report and no one can tell which subagent introduced the error. What's missing?

- A. Skip logging for subagents since only the final result matters.
- B. ✓ Log each subagent's task prompt, tool calls, and result with a shared trace ID linking them to the parent task.
- C. Log only the coordinator's final synthesized output.
- D. Rely on the model to self-report what it did in plain text.

Correct answer: B. Without per-subagent traceability (task prompt, tool calls, and result, linked by a shared trace ID), diagnosing which component introduced an error in a multi-agent pipeline becomes guesswork.

Q124. A multi-agent customer-support pipeline occasionally produces an incorrect final report and no one can tell which subagent introduced the error. What's missing?

- A. ✓ Log each subagent's task prompt, tool calls, and result with a shared trace ID linking them to the parent task.
- B. Log only the coordinator's final synthesized output.
- C. Rely on the model to self-report what it did in plain text.
- D. Skip logging for subagents since only the final result matters.

Correct answer: A. Without per-subagent traceability (task prompt, tool calls, and result, linked by a shared trace ID), diagnosing which component introduced an error in a multi-agent pipeline becomes guesswork.

Q125. A multi-agent code-review pipeline occasionally produces an incorrect final report and no one can tell which subagent introduced the error. What's missing?

- A. Log only the coordinator's final synthesized output.
- B. Rely on the model to self-report what it did in plain text.
- C. Skip logging for subagents since only the final result matters.
- D. ✓ Log each subagent's task prompt, tool calls, and result with a shared trace ID linking them to the parent task.

Correct answer: D. Without per-subagent traceability (task prompt, tool calls, and result, linked by a shared trace ID), diagnosing which component introduced an error in a multi-agent pipeline becomes guesswork.

Q126. A multi-agent compliance-check pipeline occasionally produces an incorrect final report and no one can tell which subagent introduced the error. What's missing?

- A. Rely on the model to self-report what it did in plain text.
- B. Skip logging for subagents since only the final result matters.
- C. ✓ Log each subagent's task prompt, tool calls, and result with a shared trace ID linking them to the parent task.
- D. Log only the coordinator's final synthesized output.

Correct answer: C. Without per-subagent traceability (task prompt, tool calls, and result, linked by a shared trace ID), diagnosing which component introduced an error in a multi-agent pipeline becomes guesswork.

Q127. A task involving formatting a casual internal summary is mildly ambiguous but the cost of a wrong assumption is low. How should the agent handle it?

- A. Refuse the task entirely until every detail is fully specified.
- B. ✓ State the most reasonable assumption explicitly and proceed, unless the ambiguity is severe enough that guessing wrong is costly.
- C. Always stop and ask a clarifying question before proceeding, regardless of stakes.
- D. Always guess silently with no stated assumption, to avoid slowing the user down.

Correct answer: B. For low-stakes ambiguity, stating a reasonable assumption and proceeding keeps the system useful; reserving clarifying questions for cases where a wrong guess would be genuinely costly.

Q128. A task involving choosing a default chart color scheme is mildly ambiguous but the cost of a wrong assumption is low. How should the agent handle it?

- A. ✓ State the most reasonable assumption explicitly and proceed, unless the ambiguity is severe enough that guessing wrong is costly.
- B. Always stop and ask a clarifying question before proceeding, regardless of stakes.
- C. Always guess silently with no stated assumption, to avoid slowing the user down.
- D. Refuse the task entirely until every detail is fully specified.

Correct answer: A. For low-stakes ambiguity, stating a reasonable assumption and proceeding keeps the system useful; reserving clarifying questions for cases where a wrong guess would be genuinely costly.

Q129. A task involving picking a reasonable default date range for a report is mildly ambiguous but the cost of a wrong assumption is low. How should the agent handle it?

- A. Always stop and ask a clarifying question before proceeding, regardless of stakes.
- B. Always guess silently with no stated assumption, to avoid slowing the user down.
- C. Refuse the task entirely until every detail is fully specified.
- D. ✓ State the most reasonable assumption explicitly and proceed, unless the ambiguity is severe enough that guessing wrong is costly.

Correct answer: D. For low-stakes ambiguity, stating a reasonable assumption and proceeding keeps the system useful; reserving clarifying questions for cases where a wrong guess would be genuinely costly.

Q130. A task involving selecting a default sort order for a list is mildly ambiguous but the cost of a wrong assumption is low. How should the agent handle it?

- A. Always guess silently with no stated assumption, to avoid slowing the user down.
- B. Refuse the task entirely until every detail is fully specified.
- C. ✓ State the most reasonable assumption explicitly and proceed, unless the ambiguity is severe enough that guessing wrong is costly.
- D. Always stop and ask a clarifying question before proceeding, regardless of stakes.

Correct answer: C. For low-stakes ambiguity, stating a reasonable assumption and proceeding keeps the system useful; reserving clarifying questions for cases where a wrong guess would be genuinely costly.

Q131. A subagent responsible for a single narrow fact lookup is given the same large context budget as the coordinator's full session. What's the issue?

- A. Size all subagent budgets identically regardless of subtask complexity.
- B. ✓ Size the subagent's context budget to what that specific subtask needs, not to the whole system's total budget.
- C. Give every subagent the same context budget as the coordinator by default.
- D. Give subagents an unlimited context budget to be safe.

Correct answer: B. Context budgets should be scoped to what a given subagent's task actually requires; over-provisioning wastes cost and can dilute the subagent's focus with irrelevant capacity.

Q132. A subagent responsible for summarizing one short document is given the same large context budget as the coordinator's full session. What's the issue?

- A. ✓ Size the subagent's context budget to what that specific subtask needs, not to the whole system's total budget.
- B. Give every subagent the same context budget as the coordinator by default.
- C. Give subagents an unlimited context budget to be safe.
- D. Size all subagent budgets identically regardless of subtask complexity.

Correct answer: A. Context budgets should be scoped to what a given subagent's task actually requires; over-provisioning wastes cost and can dilute the subagent's focus with irrelevant capacity.

Q133. A subagent responsible for validating one data field is given the same large context budget as the coordinator's full session. What's the issue?

- A. Give every subagent the same context budget as the coordinator by default.
- B. Give subagents an unlimited context budget to be safe.
- C. Size all subagent budgets identically regardless of subtask complexity.
- D. ✓ Size the subagent's context budget to what that specific subtask needs, not to the whole system's total budget.

Correct answer: D. Context budgets should be scoped to what a given subagent's task actually requires; over-provisioning wastes cost and can dilute the subagent's focus with irrelevant capacity.

Q134. A subagent responsible for answering one yes/no sub-question is given the same large context budget as the coordinator's full session. What's the issue?

- A. Give subagents an unlimited context budget to be safe.
- B. Size all subagent budgets identically regardless of subtask complexity.
- C. ✓ Size the subagent's context budget to what that specific subtask needs, not to the whole system's total budget.
- D. Give every subagent the same context budget as the coordinator by default.

Correct answer: C. Context budgets should be scoped to what a given subagent's task actually requires; over-provisioning wastes cost and can dilute the subagent's focus with irrelevant capacity.

Q135. A long-running multi-day research project loses track of which subtasks are already complete after several turns. What's missing?

- A. Persist state only in the model's own summarized memory of the conversation.
- B. ✓ Persist an explicit task list or plan object that survives across turns, rather than relying on the model to reconstruct state from conversation history alone.
- C. Rely entirely on the model re-deriving the plan from scratch each turn.
- D. Store no state at all and treat every turn as fully independent.

Correct answer: B. An explicit, persisted task-list or plan object gives the system a reliable source of truth for progress, instead of depending on the model to reconstruct state from a long conversation history.

Q136. A long-running multi-step migration task loses track of which subtasks are already complete after several turns. What's missing?

- A. ✓ Persist an explicit task list or plan object that survives across turns, rather than relying on the model to reconstruct state from conversation history alone.
- B. Rely entirely on the model re-deriving the plan from scratch each turn.
- C. Store no state at all and treat every turn as fully independent.
- D. Persist state only in the model's own summarized memory of the conversation.

Correct answer: A. An explicit, persisted task-list or plan object gives the system a reliable source of truth for progress, instead of depending on the model to reconstruct state from a long conversation history.

Q137. A long-running extended data-cleanup job loses track of which subtasks are already complete after several turns. What's missing?

- A. Rely entirely on the model re-deriving the plan from scratch each turn.
- B. Store no state at all and treat every turn as fully independent.
- C. Persist state only in the model's own summarized memory of the conversation.
- D. ✓ Persist an explicit task list or plan object that survives across turns, rather than relying on the model to reconstruct state from conversation history alone.

Correct answer: D. An explicit, persisted task-list or plan object gives the system a reliable source of truth for progress, instead of depending on the model to reconstruct state from a long conversation history.

Q138. A long-running long-running audit task loses track of which subtasks are already complete after several turns. What's missing?

- A. Store no state at all and treat every turn as fully independent.
- B. Persist state only in the model's own summarized memory of the conversation.
- C. ✓ Persist an explicit task list or plan object that survives across turns, rather than relying on the model to reconstruct state from conversation history alone.
- D. Rely entirely on the model re-deriving the plan from scratch each turn.

Correct answer: C. An explicit, persisted task-list or plan object gives the system a reliable source of truth for progress, instead of depending on the model to reconstruct state from a long conversation history.

End of Domain 1 deep-dive. This is an independent study aid; verify current exam logistics and blueprint details against your Claude Partner Network portal before sitting the exam.